



Tecnológico de Monterrey

Estudiantes:

Axel Javier Rosas Rodríguez | A01738607

Elian Cantalapiedra Sabugal | A01738462

Edwin Emmanuel Salazar Meza | A01738380

Profesor:

Rigoberto Cerino Jiménez

Nombre de la Institución:

Instituto Tecnológico y de Estudios Superiores de
Tecnológico de Monterrey

Materia:

Implementación de Robótica Inteligente

Fecha:

17 de mayo de 2026

Título del trabajo:

Reto semanal 6. Manchester Robotics

Resumen

Este reporte contiene el desarrollo de un sistema de navegación autónomo para el robot Puzzlebot, cumpliendo con el reto propuesto por Manchester Robotics. La solución que implementamos integra una capa de visión artificial para el seguimiento de líneas y una capa de decisión para la gestión del semáforo. Nuevamente utilizamos ROS 2 para poder coordinar nuestros tres nodos principales, un procesador de visión (follower_node), un estimador de odometría (odometry_robot_node) y un controlador de lazo cerrado (lazo_cerrado_node). Se diseñó el algoritmo para que fuera robusto ante cambios de iluminación ya que era uno de los mayores problemas, así como para la pérdida de trayectoria, siguiendo la lógica de banderas de tráfico (Red, Yellow, Green) y operando de forma totalmente autónoma sin nuestra intervención.

Objetivos

Objetivo General:

- Añadir una capa de seguimiento de línea a la capa de decisión y navegación punto a punto previamente desarrollada, utilizando la cámara del Puzzlebot.

Objetivos específicos:

- Desarrollar un algoritmo robusto de seguimiento de línea para un circuito diseñado por nosotros.
- Implementar la lógica de comportamiento ante semáforos
 - Rojo indicaba un paro total.
 - Amarillo indicaba reducir la velocidad.
 - Verde indicaba continuar con la trayectoria.

Introducción

El uso de visión artificial para el seguimiento de líneas es un problema clásico en la robótica móvil que requiere la integración de percepción y control. Manchester nos pide crear un sistema robusto, definiendo robustez como la capacidad que nuestro sistema mantenga un desempeño aceptable, no saliéndose de la pista y respetando el semáforo, a pesar de los cambios de iluminación, perturbaciones externas como la pérdida momentánea de las líneas en curvas cerradas o cambios bruscos de trayectorias

- Conceptos Relevantes
 - Procesamiento en tarjeta embebida: El Puzzlebot utiliza una arquitectura de procesamiento llamado NVIDIA Jetson, que le permite ejecutar algoritmos de OpenCV en tiempo real sobre mensajes de ROS 2.

- Interconexión Jetson-Cámara: Utilizamos el launch de manchester para lanzar la cámara, lo que nos permite suscribirnos al tópic (`/video/source/raw`) y que nuestro nodo follower procese los frames usando CvBridge.
- Robustez en Sistema de Visión: Se implementó mediante el uso de espacios de color HSV que son menos sensibles al brillo comparados al RGB, esto para trabajar con el semáforo y los filtros morfológicos para las líneas.
- Seguimiento de trayectorias: Basado en el cálculo de error de la posición del carril respecto al centro óptico de la cámara, lo que alimenta un controlador de lazo cerrado.

Solución del problema

Metodología y Algoritmos Implementados

Se siguieron las indicaciones de solo usar librerías estándar como OpenCV y NumPy, para la creación de nuestro sistema.

1. Percepción y Visión Artificial (follower)

Para lograr la robustez necesaria, el nodo de visión segmenta el entorno en dos regiones de interés (ROI)

- ROI Inferior : Esta está enfocada en el seguimiento de línea, se aplica una máscara trapezoidal estática, lo que le permite ignorar el entorno fuera de la pista. La imagen se convierte a escala de grises, después un filtro de enfoque, un filtro Gaussiano para eliminar ruido, y por último una umbralización binaria invertida. Usamos el algoritmo de Canny y la Transformada de Hough Lineas Proporcional (HoughLinesP) para extraer los bordes. Clasificamos las líneas por su ángulo y mediante una regresión polinomial (`np.polyfit`) se calcula el centro del carril. El error se define como:

$$error = centro_imagen - centro_carril$$

```

# — ROI inferior: seguimiento de línea —————
offset_y = int(height * 0.5)
roi      = img[offset_y:, :]
roi_height, roi_width = roi.shape[:2]

# Aplicar máscara trapezoidal
roi_masked = cv2.bitwise_and(roi, roi, mask=self.mask)

# Preprocesamiento
gray      = cv2.cvtColor(roi_masked, cv2.COLOR_BGR2GRAY)
sharpened = cv2.filter2D(gray, -1, self.sharpen_kernel)
blurred   = cv2.GaussianBlur(sharpened, (9, 9), 0)
_, binary_inv = cv2.threshold(blurred, 100, 255, cv2.THRESH_BINARY_INV)

# Operaciones morfológicas
morph = cv2.erode(binary_inv, self.kernel, iterations=2)
morph = cv2.dilate(morph, self.kernel, iterations=2)

# Detección de bordes
edges = cv2.Canny(morph, 50, 150, apertureSize=3)

# Hough Lines Probabilístico
lines = cv2.HoughLinesP(edges, 1, np.pi / 180,
                        threshold=30,
                        minLineLength=15,
                        maxLineGap=100)

# Clasificación de líneas por ángulo
left_lines, right_lines = [], []
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        angle = np.degrees(np.arctan2(y2 - y1, x2 - x1))
        if abs(angle) < 10:
            continue
        (left_lines if angle < 0 else right_lines).append(line)

# Cálculo del centro del carril mediante regresión
centro_imagen = roi_width // 2
resultado_izq = self.promedio_linea(left_lines, roi_height)
resultado_der = self.promedio_linea(right_lines, roi_height)

x_izq = resultado_izq[0] if resultado_izq is not None else 0
x_der = resultado_der[0] if resultado_der is not None else roi_width

centro_carril = (x_izq + x_der) // 2

# Error lateral
error = float(centro_imagen - centro_carril)

```

Imagen 1 – Seguimiento de línea (ROI inferior)

- ROI superior: Ahora se enfoca en la detección del semáforo, extraemos el 60% superior del cuadrado, y se convierte a HSV. Utilizamos la función (cv2.inRange) para definir umbrales específicos para cada estado (rojo,

amarillo y verde). El algoritmo se enfoca en encontrar contornos y filtrarlos por un mínimo de área de 170 píxeles, para que el robot reaccione solo a una distancia cercana.

```
# — ROI superior: detección de semáforo —————
roi_sup_h    = int(height * 0.6)    # 60% superior de la imagen
roi_superior = img[:roi_sup_h, :]

# Conversión a HSV
hsv = cv2.cvtColor(roi_superior, cv2.COLOR_BGR2HSV)

estado_detectado = 'NINGUNO'

for color, rangos in self.config_hsv.items():
    mask_color = np.zeros(hsv.shape[:2], dtype=np.uint8)

    # Umbrales HSV por color
    for lower, upper in rangos:
        mask_color = cv2.add(
            mask_color,
            cv2.inRange(hsv, np.array(lower), np.array(upper))
        )

    # Extracción de contornos
    contours, _ = cv2.findContours(
        mask_color, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    if not contours:
        continue

    # Contorno de mayor área
    c = max(contours, key=cv2.contourArea)
    area = cv2.contourArea(c)

    # Filtro por área mínima
    if area > 170:
        estado_detectado = color
        break
```

Imagen 2 – Detección de semáforo (ROI superior)

2. Estimación de Estados y Filtrado Odometry)

Para mitigar el ruido en los sensores, en el nodo de odometría se implementó un filtro de promedio móvil exponencial (Filtro Alfa) sobre las velocidades de las ruedas:

$$w_{filtrada} = \alpha * w_{actual} + (1 - \alpha) * w_{anterior}$$

Con un $\alpha = 0.85$, se suavizan los picos de los encoders. Posteriormente, se aplican las ecuaciones de la cinemática diferencial para integrar en el tiempo la posición X, Y y el ángulo θ , transformando este último a cuaterniones mediante la librería `transforms3d`.

```
# — Filtro Alfa ( $\alpha = 0.85$ ) —————
self.wr_f = (0.85 * msg_r.data) + (1.0 - 0.85) * self.wr_f
self.wl_f = (0.85 * msg_l.data) + (1.0 - 0.85) * self.wl_f

# — Cinemática diferencial —————
v_r = self._r * self.wr_f
v_l = self._r * self.wl_f

V      = 0.5 * (v_r + v_l)
Omega  = (1.0 / self._l) * (v_r - v_l)

# — Integración de la pose —————
self.Th += Omega * dt
self.Th = np.arctan2(np.sin(self.Th), np.cos(self.Th))
self.X  += V * np.cos(self.Th) * dt
self.Y  += V * np.sin(self.Th) * dt

# — Conversión a cuaterniones —————
q = transforms3d.euler.euler2quat(0, 0, self.Th)
```

Imagen 3 – Odometría con filtro alfa

3. Control de Lazo Cerrado y Capa de decisión (Controller)

Este nodo es el cerebro del robot, el cual implementa lo solicitado por manchester, la lógica de banderas mediante una máquina de estados finitos:

- Lógica de Banderas:
 - Green Flag: Velocidad nominal de referencia ($v_{ref} = 0.1 \text{ m/s}$)
 - Yellow Flag: Velocidad reducida al 50% ($v_{ref} = 0.05 \text{ m/s}$) para aproximación segura

- Red Flag: Velocidad cero ($v_{ref} = 0.0 \text{ m/s}$) generando un parto total controlado.

```
# — Lógica de banderas (semáforo) —————
if self.ultimo_estado == 'ROJO':
    v_ref = 0.0 # Red Flag → paro total
elif self.ultimo_estado == 'AMARILLO':
    v_ref = self.v_ref * 0.5 # Yellow Flag → 50% de velocidad
else:
    v_ref = self.v_ref # Green Flag → velocidad nominal (0.1 m/s)
```

Imagen 4 – Lógica de banderas

- Estrategia de Recuperación: Cuando line_detected es verdadero, el robot avanza y corrige su giro con un control proporcional basado en el error en pixeles.
 - Estado SEARCHING: En cambio si la línea se pierde por perturbaciones del entorno, la máquina cambia de estado automáticamente. El robot reduce su velocidad a un mínimo seguro (0.03 m/s) y comienza a girar en zigzag, invirtiendo su dirección angular cada segundo ($\text{search_period} = 1.0$). Este barrido lateral continúa de forma autónoma hasta que el nodo de visión vuelve a la línea de la pista.

```

# — Estrategia de recuperación de línea —————
if self.line_detected:
    self.state = TRACKING
    self.search_timer = 0.0
else:
    self.state = SEARCHING

# — Estado SEARCHING (zigzag) —————
if self.state == SEARCHING:
    self.search_timer += self.dt
    if self.search_timer ≥ self.search_period:
        self.search_timer = 0.0
        self.search_dir *= -1      # invertir dirección angular

    cmd.linear.x = 0.03            # velocidad mínima segura
    cmd.angular.z = self.search_dir * self.w_search

```

Imagen 5 – Estrategia de recuperación

- Controladores PI de Velocidad: Para asegurar el seguimiento de las referencias deseadas frente a las no linealidades físicas de los motores, se programaron lazos Proporcional-Integral (PI) internos con saturación integral (`np.clip`) para evitar el fenómeno de Windup

```

# — Controladores PI de velocidad con anti-windup —————
def control_linear(self, v_ref):
    e = v_ref - self.v_real
    self.e_v_int += e * self.dt
    self.e_v_int = np.clip(self.e_v_int, -0.5, 0.5)  # anti-windup
    u = self.Kp_v * e + self.Ki_v * self.e_v_int
    return float(np.clip(u, -0.3, 0.3))

def control_angular(self):
    w_ref = np.clip(self.error_pixeles * self.Kp_vision, -2.5, 2.5)
    error = w_ref - self.w_real
    self.e_w_int += error * self.dt
    self.e_w_int = np.clip(self.e_w_int, -0.5, 0.5)  # anti-windup
    w = self.Kp_w * error + self.Ki_w * self.e_w_int
    return float(np.clip(w, -3.0, 3.0))

```

Imagen 6 – Controladores PI

Resultados

El sistema de navegación autónoma implementado cumplió satisfactoriamente con los objetivos planteados en el mini reto. El robot fue capaz de seguir la línea del circuito de manera continua, manteniéndose dentro de la pista incluso ante variaciones moderadas de iluminación y pequeñas perturbaciones externas.

La segmentación por ROI permitió reducir significativamente el ruido visual y mejorar la estabilidad del cálculo del centro del carril. El uso del espacio de color HSV para la detección del semáforo resultó efectivo, logrando una clasificación confiable de los estados rojo, amarillo y verde, siempre que el semáforo se encontrara dentro del rango de distancia definido por el umbral de área mínima.

La lógica de banderas fue ejecutada correctamente:

En estado verde, el robot avanzó a la velocidad nominal sin desviaciones importantes.
En amarillo, la reducción de velocidad permitió una aproximación segura al semáforo.
En rojo, el robot realizó un paro total controlado sin oscilaciones.

Cuando la línea se perdió momentáneamente, la estrategia de recuperación tipo búsqueda en zigzag permitió retomar la trayectoria de forma autónoma en la mayoría de los casos, sin intervención externa. Los controladores PI internos mostraron un comportamiento estable, con un seguimiento adecuado de las referencias de velocidad y sin evidenciar problemas de windup gracias a la saturación integral implementada.

Conclusiones

Se logró desarrollar un sistema de navegación autónoma que integra percepción, decisión y control utilizando ROS 2, cumpliendo con los requerimientos del mini reto. La separación del sistema en nodos independientes facilitó el diseño, la depuración y la escalabilidad de la solución.

El uso de visión artificial basada en opencv, junto con técnicas de filtrado y segmentación adecuadas, demostró ser una herramienta eficaz para el seguimiento de líneas y la detección de semáforos en tiempo real sobre una plataforma embebida. Asimismo, la implementación de una máquina de estados finitos permite una gestión clara y ordenada de los distintos comportamientos del robot.

Finalmente, la estrategia de recuperación ante la pérdida de línea incrementó notablemente la autonomía del sistema, reforzando la definición de robustez solicitada en el reto. Como trabajo futuro, se podrían explorar técnicas adaptativas para los umbrales de visión o el uso de controladores más avanzados para mejorar el desempeño en escenarios con iluminación extrema o curvas más cerradas.

Referencias

Manchester Robotics (MCR2). Challenges: Line Following Mini Challenge.